Cascade Tuning Guide

Team 936C ▪ VEX V5RC *Override* 2026–27

Getting the lift to score reliably, one number at a time

Read this first

Do the steps **in order**. Each one depends on the one before it – there is no point tuning the control loop if the inches are wrong, and no point measuring heights if the lift does not home.

- **Step 1** – Make it home correctly
- **Step 2** – Make the inches true *(most important)*
- **Step 3** – Make it move smoothly
- **Step 4** – Make the heights right
- **Step 5** – Full driver check

1 How the cascade works

Understanding this chain makes the rest of the guide obvious.

1. The V5 motor has a built-in **encoder**. It counts how far the motor has turned, in degrees.
2. The code converts degrees into **inches** using one number: `IN_PER_MOTOR_DEG`.
3. When you press **R1**, the code looks up the **target height** for the current goal class and rung.
4. A **control loop** drives the motor until the measured height matches the target.
5. Then it deposits the Pin or Cup, and on a confirmed placement it advances the rung.

That gives exactly four things to get right:

What	What it means	If it is wrong
Zero	Where measuring starts (homing)	Every height is shifted by the same amount
Scale	How many inches one motor degree is (`IN_PER_MOTOR_DEG`)	Heights are wrong by a percentage – worse the higher you go
Feel	How the control loop moves (KP, KD, etc.)	It shakes, sags, jerks, or never settles
Targets	How high each rung is	It reaches the wrong height and misses the Goal

The one-sentence version

Homing sets the *starting point*, `IN_PER_MOTOR_DEG` sets the *scale*, the PD gains set the *smoothness*, and the preset table sets the *destinations*.

2 Before you start

You need:

- The robot built, with the cascade moving freely by hand
- A **rigid hard stop** at the bottom of travel, ideally with a **rubber bumper**
- A **tape measure**
- A charged battery
- One hand free to **cut power** the instant something goes wrong

What "cut power" means

I am not talking about a special switch – VEX robots do not have one. The fastest way to stop the robot is the **power button on the V5 Brain**. Pressing it turns the Brain off, which stops your program and the motors immediately. Pulling the **battery connector** does the same thing.

Safety

Test small movements first. The code clamps targets to `MAX_IN` (30 in), but if `IN_PER_MOTOR_DEG` is badly wrong the lift can still travel further than you expect. Keep a hand on the V5 Brain's power button so you can cut power instantly.

2.1 The buttons you will use while tuning

Button	What it does
R1	Score at the current rung, then move up one rung
R2	Score again at the *same* rung (no advance)
X	New stack – sets rung to 0 ("this Goal is empty")
Y	Change goal class: ALLY → SHORT → TALL
Left / Right	Rung −1 / +1 (manual sync)
L1 / L2	Jog the lift up / down by hand (hold)
A	Re-home the lift

The controller LCD shows the current state, for example `SHORT rung 2/5 CUP`. That tells you the goal class, how many objects are already placed, and whether the next one is a Pin or a Cup.

3 The only file you edit

Every number in this guide lives in one place:

```
include/override/constants.hpp -> namespace constants::lift { ... }
```

Do not edit anything else to tune

The preset table is *generated* from these constants, so you normally change two or three numbers instead of fifteen. Only if a single cell is stubborn do you add an override in `cascade.cpp` (explained in Step 4).

4 Step 1 — Make it home correctly

The lift uses **hard-stop homing**. At boot it drives slowly downward. When the encoder stops turning for 200 ms, the code decides the lift is sitting on the stop, calls that position **zero**, and lifts to travel height.

4.1 Do this

1. Power the robot on with the cascade somewhere in the middle of its travel.
2. Watch what it does. It should drive **down**, settle on the stop, then rise a little to the travel height (1 inch).
3. Power-cycle and repeat once more to confirm it does the same thing both times.

4.2 What you should see

What happens	What to change
Drives up at boot	Flip the sign of `HOMING_VOLTAGE` (change `-40` to `40`)
Homes correctly	Nothing – go to Step 2
Sits still, never moves, then gives up after 4 s	The motor is too weak to move at that voltage. Raise it: `HOMING_VOLTAGE` from `-40` to `-60`. Also check for mechanical binding
Homes too early, before reaching the bottom	False stall. Raise `HOMING_VOLTAGE` magnitude so it pushes through; also check nothing is dragging
Drives all the way down, keeps grinding, then gives up	It is not detecting the stop. Make the stop rigid; a soft or springy stop lets the motor keep creeping
Boot takes too long	The full travel takes longer than `HOMING_TIMEOUT_MS` (4 s). Raise it

Why a rigid stop matters

The stop *is* your zero. If it flexes, compresses, or squishes a different amount each time, your zero moves and every height drifts with it. Use a solid stop, and put a rubber bumper on it to protect your string or chain from the stall.

5 Step 2 — Make the inches true

This is the most important number in the whole subsystem. Skip it and nothing else will be accurate.

5.1 Why it matters

The loop drives the motor until the *measured* height matches the target. If your scale is wrong, the lift confidently goes to the wrong place.

Worked example

Suppose the real value is `0.045`, but the code is set to `0.030`.
You command **6 inches**. The code works out it needs `6 / 0.030 = 200` motor degrees. But 200 degrees is really `200 * 0.045 = 9 inches`.
So the lift goes to **9 inches** instead of 6. And the error gets worse the higher you go.

5.2 How to measure it (no extra tools needed)

R1 already commands a known height, so you can use it as a measuring stick.

1. Let the robot home.
2. Press **Y** until the LCD says `SHORT`.
3. Press **X** so the rung is 0 (the LCD shows `rung 0/5`).

4. Press **R1**. The lift moves to what it *thinks* is **6.30 inches**.
5. Measure the **actual** height with your tape measure. Call it M.

Pick one fixed reference point on the carriage – the bottom edge of the end effector is a good choice – and always measure that same point from the floor.

5.3 Now do the maths

```
new IN_PER_MOTOR_DEG = 0.030 * ( M / 6.30 )
```

Worked example

You measure $M = 4.20$ inches.
`0.030 * (4.20 / 6.30)= 0.020`
So you change `IN_PER_MOTOR_DEG` from `0.030` to `0.020`.

Rule of thumb: if it goes *higher* than expected, your number is too *small*. If it goes *lower*, too *big*. The formula handles both.

5.4 Check your work

1. Re-upload and repeat the measurement.
2. It should now be within about a quarter inch.
3. Test a *second* height to be sure – press **R1** again for rung 2 (9.55 in) and measure.
4. If the small height is right but the tall one is off, your scale is still wrong. Repeat the calculation using the taller measurement, which gives a more accurate result.

6 Step 3 — Make it move smoothly

Now that the numbers are true, adjust how the lift *feels*. All of these are in `constants::lift`.

6.1 What each number does

Constant	Default	What it does
`KP`	14.0	How hard it pushes toward the target. Higher = faster and stiffer, too high = shaking
`KD`	1.2	Damping – acts like a predictive brake. Higher = less overshoot
`GRAVITY_FF`	6.0	Constant upward push so the motor does not fight gravity on error alone. Raise it if the lift sags
`MAX_VOLTAGE`	100.0	The output ceiling. Caps top speed
`SLEW_V_PER_S`	600.0	Limits how fast the output can change – limits acceleration. Lower = gentler starts
`TOL_IN`	0.30	"Close enough" band, in inches

There is no integral term, and that is deliberate

This is a **PD controller plus gravity feed-forward**, not a full PID. An integral term mostly causes wind-up and overshoot on a lift, and you do not need it anyway: once the lift is within `TOL_IN` of the target, the code stops the motors and lets the **HOLD brake** hold position mechanically. If the lift ever sags more than twice the tolerance, the loop automatically re-engages.

6.2 Do this

1. Set the goal class to `SHORT` and the rung to 0.
2. Press **R1** and watch the lift climb to 6.30 inches.
3. Press **R2** a few times so it repeats the same move.
4. Watch for the symptoms below and change **one** number at a time.

6.3 Symptom to fix

What you see	What to change
Shakes or hunts around the height	Lower `KP` (14 → 9), or raise `KD` (1.2 → 2.5)
Slow to get there	Raise `KP` (14 → 22)
Overshoots, then comes back	Raise `KD` (1.2 → 2.5), or lower `SLEW_V_PER_S`
Sags below the target while rising	Raise `GRAVITY_FF` (6 → 12)
Jerks or lurches when it starts	Lower `SLEW_V_PER_S` (600 → 300)
Never reports “arrived”	Widen `TOL_IN` (0.30 → 0.50)

Two rules

1. Change one number, then retest. Changing three at once tells you nothing.
2. If it never settles, it will still not hang: the score command has a 1.5 second approach timeout, so a bad tune degrades into a slower score rather than a frozen robot.

7 Step 4 — Make the heights right

The fifteen heights are **generated** from three numbers, not typed in one by one:

```
release_height = goal_rim + (rung − 1) ∗ STEP_IN + CLEARANCE_IN
```

So you tune the *generator*. Here are the values it produces today (these are guesses based on the field, not measured truth):

Rung	Object	ALLY	SHORT	TALL
1	Pin	3.75 in	6.30 in	9.20 in
2	Cup	7.00 in	9.55 in	12.45 in
3	Pin	10.25 in	12.80 in	15.70 in
4	Cup	13.50 in	16.05 in	18.95 in
5	Pin	16.75 in	19.30 in	22.20 in

7.1 4a — Get rung 1 right

1. Put a real Goal on the floor.
2. Press **Y** to select the goal class, then **X** to set rung 0.
3. Press **R1** and watch where the Pin drops.
4. If it is too low or too high, nudge the lift with **L1**/**L2** to find the spot where it seats cleanly.
5. Note how far off you were, then adjust `CLEARANCE_IN` (0.50 in) up or down by that amount.
6. Repeat for the other two goal classes. If one class is off by a different amount from the others, adjust its rim instead: `GOAL_ALLIANCE_IN`, `GOAL_NEUTRAL_IN`, or `GOAL_TALL_IN`.

7.2 4b — Measure the real step

This is the number that decides how much taller each object sits.

1. Stack objects on a Goal until you have something at rung 1 and something at rung 3 (two Pins).
2. Measure the height difference between the two seat points.
3. That difference is **two** steps, so divide it by two.
4. Set `STEP_IN` to that result.

Why 3.25 is the starting guess

A Pin and a Cup are each 6.5 inches tall and have two halves. The rules require each object to nest halfway into the one below, so in theory each added object raises the stack by one half – 3.25 inches. Real objects do not seat perfectly, so **measure yours** and put the real number in.

7.3 4c — Override a single stubborn cell

If one specific rung is consistently off while the others are fine, you can overwrite just that cell. Add a line at the end of the `Cascade` constructor in `src/override/subsystems/cascade.cpp`, right after the seeding loop:

```
// after the seeding loop in Cascade::Cascade()
presets_[1][2] = 9.80; // [SHORT][rung 2 Cup] – measured
presets_[1][4] = 16.20; // [SHORT][rung 4 Cup] – measured
```

The first index is the goal class: `0` = ALLY, `1` = SHORT, `2` = TALL. The second is the rung (1–5).

Rung 1 is special

Rung 1 is only valid on an **empty** Goal, because a Goal can hold exactly one Pin half. The code cannot tell whether a Goal is empty – pressing **X** is how you tell it. If you press X at a Goal that already has a Pin in it, rung 1 will command a height where the opening is already occupied and the Pin will not seat.

8 Step 5 — Full driver check

Run through this once with the robot on the floor and real Goals.

Test	Expected result
Boot	Lift homes on its own, then sits at travel height. No grinding.
X then R1	Places one Pin, then the LCD rung goes 0 → 1.
R1 again	Places a Cup at rung 2 (the LCD says `CUP` beforehand), rung goes to 2.
R2	Repeats the same rung without advancing.
Left / Right	Rung number changes up and down.
L1 / L2	Lift jogs smoothly and holds position when released.
A	Lift re-homes from wherever it is.
Y	Cycles ALLY → SHORT → TALL and the height changes.

Do not hold L2 at the bottom
With hard-stop homing there is no switch to stop you, so holding jog-down will stall the motors against the stop. Tap it, do not lean on it.

9 Troubleshooting

Symptom	Likely cause	Fix
Nothing moves at all; R1 just rumbles	Homing failed, so everything is locked out on purpose	Check the boot behaviour using the Step 1 table
Drives up at boot	Homing direction reversed	Flip the sign of `HOMING_VOLTAGE`
Homes at the wrong place every time	Stop is not repeatable	Make the stop rigid; add a rubber bumper
Reaches a height, waits, and never settles	`TOL_IN` too tight, or `KP` too low	Widen `TOL_IN`; raise `KP`
Always goes too high or too low by a percentage	`IN_PER_MOTOR_DEG` wrong	Redo Step 2
Everything is shifted by the same amount	Zero is off	Check the stop, or set `ZERO_OFFSET_IN`
The rung never advances	The intake saw a Pin that never left	Intended: press R2 to retry
Cup rungs behave oddly	The Cup manipulator has no motor yet, so cup placement cannot be confirmed	Expected for now; the code trusts the sequence
It refuses to score on the centre Goal	Endgame lockout (last 10 s)	Intended – pick another goal class
Middle of a match, heights are suddenly wrong	The lift was re-homed against a jam	Check for something blocking the cascade

10 Quick reference — every constant

Constant	Default	Notes
Geometry		
`PIN_HEIGHT_IN`	6.5	Manual value, do not change
`CUP_HEIGHT_IN`	6.5	Manual value, do not change
`HALF_IN`	3.25	One Pin/Cup half
`GOAL_ALLIANCE_IN`	3.25	Alliance Goal rim
`GOAL_NEUTRAL_IN`	5.80	Neutral short Goal rim
`GOAL_TALL_IN`	8.70	Centre Goal rim
`STEP_IN`	3.25	**Measure this** – rise per object
`CLEARANCE_IN`	0.50	**Tune** – slack above the seat
`TRAVEL_IN`	1.00	Height while driving
`MAX_IN`	30.00	Software ceiling, keep under 50 in
`MAX_RUNG`	5	Highest rung to command
Scale and homing		
`IN_PER_MOTOR_DEG`	0.030	**Calibrate first – Step 2**
`ZERO_OFFSET_IN`	0.0	Only if the stop is not the true zero
`USE_HOMING_SWITCH`	false	`true` to use a limit switch on ADI C instead
`HOMING_VOLTAGE`	−40	Slow descent speed; flip sign if it drives up
`HOMING_MIN_MS`	350	Ignore “not moving” before this
`HOMING_STALL_DETECT_MS`	200	No encoder motion this long = seated
`HOMING_TIMEOUT_MS`	4000	Give up after this
`LIMIT_ACTIVE_HIGH`	true	Only used if the switch is enabled
Control loop		
`KP`	14.0	**Tune** – push toward target
`KD`	1.2	**Tune** – damping
`GRAVITY_FF`	6.0	**Tune** – upward bias
`MAX_VOLTAGE`	100.0	Output ceiling
`SLEW_V_PER_S`	600.0	Acceleration limit
`TOL_IN`	0.30	“Close enough” band
`CONTROL_PERIOD_MS`	10	100 Hz – leave alone
`JOG_VOLTAGE`	40	L1/L2 jog speed

If you only remember three things

1. `IN_PER_MOTOR_DEG` decides whether heights mean anything. Calibrate it first.
2. The hard stop *is* your zero – it must be rigid and repeatable.
3. Change one number at a time, then retest.